Architecture and Engineering of an Automotive Dealership ERP Engine and Interactive E-Commerce Platform

By

Ibrahim Hasan

Student ID: **2110316**

Summer 2026

Academic Supervisor:

Azwad Abid

Lecturer

Department of Computer Science & Engineering

Independent University, Bangladesh

Industry Supervisor:

Ayan Chowdhury

Senior Officer, ICT Wing (MIS)

Radiant Pharmaceuticals Limited

August 16, 2026

Dissertation submitted in partial fulfillment for the degree of Bachelor of Science in Computer Science & Engineering

Department of Computer Science & Engineering

Independent University, Bangladesh

Attestation

I confirm that the technical work, architectural designs, code implementations, and analytical benchmarks presented in this report, titled **"Architecture and Engineering of an Automotive Dealership ERP Engine and Interactive E-Commerce Platform,"** represent my own independent effort. This capstone engineering project was conducted during my Summer 2026 internship term at Radiant Pharmaceuticals Limited.

All database schemas, Python/Django services, raw SQL analytical procedures, access-control configurations, and user-facing templates documented herein were authored by me, except where standard open-source dependencies or scholarly literature are formally acknowledged. Furthermore, I affirm that this submission complies with the academic integrity regulations of Independent University, Bangladesh and adheres strictly to the departmental Turnitin similarity threshold.

....

Signature of the Student Date

Ibrahim Hasan
Student ID: 2110316
Department of Computer Science & Engineering
Independent University, Bangladesh

Acknowledgement

I express my deepest appreciation to my academic supervisor, Mr. Azwad Abid, Lecturer in the Department of Computer Science & Engineering at Independent University, Bangladesh (IUB). His rigorous critiques, architectural suggestions, and methodological oversight were instrumental in elevating this project to an enterprise engineering standard.

I am immensely indebted to my industry supervisor, Mr. Ayan Chowdhury, Senior Officer within the ICT Wing (MIS) at Radiant Pharmaceuticals Limited, for granting me the autonomy to design complex database systems, test server configurations, and experience professional enterprise software lifecycles firsthand.

My sincere thanks are also extended to the Department of Computer Science & Engineering, the Internship Coordination committee, and faculty evaluation panels at IUB for structuring the CSE499 curriculum to bridge theoretical university studies with industry software execution.

Lastly, I remain profoundly grateful to my family and peers, whose steadfast patience and moral encouragement sustained my focus throughout this demanding development period.

Ibrahim Hasan
Student ID: 2110316
Department of Computer Science & Engineering
Independent University, Bangladesh

Letter of Transmittal

August 16, 2026

Azwad Abid
Lecturer
Department of Computer Science & Engineering
School of Engineering, Technology & Sciences
Independent University, Bangladesh (IUB)

Subject: Formal Submission of CSE499 Final Internship Dissertation

Dear Sir,

I hereby present my final internship dissertation, **"Architecture and Engineering of an Automotive Dealership ERP Engine and Interactive E-Commerce Platform,"** prepared in partial fulfillment of the academic requirements for the degree of Bachelor of Science in Computer Science & Engineering.

Carried out during my industry placement at Radiant Pharmaceuticals Limited under your supervision and the industry mentorship of Mr. Ayan Chowdhury, this report documents the end-to-end engineering of a dual-application automotive platform. The work encompasses relational star-schema design, raw SQL performance optimization, 9-tier RBAC authorization, and automated testing, strictly prepared in accordance with the CSE499 Outcome-Based Education (OBE) guidelines.

I appreciate the time, technical guidance, and constructive recommendations you provided over the course of this semester, and I look forward to presenting and demonstrating this system during the final viva examination.

Respectfully submitted,

Ibrahim Hasan
Student ID: 2110316
Department of Computer Science & Engineering

Independent University, Bangladesh

Evaluation Committee

Supervision Panel

.....	.....
Academic Supervisor	Industry Supervisor

Panel Members

.....	.....
Panel Member 1	Panel Member 2
.....	.....
Panel Member 3	Panel Member 4

Office Use

.....	.....
Internship Coordinator	Head of the Department

.....

Industry Coordinator of the Department

Abstract

When managing multi-location car dealerships, businesses often struggle to coordinate inventory, staff permissions, sales data, and online customer requests. Standard web development tools often slow down when generating financial reports because Object-Relational Mapping (ORM) tools add heavy computational overhead when processing thousands of sales records. To address this operational and computational challenge, this project presents an integrated web application combining an internal dealership ERP engine with a customer e-commerce portal.

The platform comprises two primary subsystems: an administrative ERP module (`car_sales`) equipped with a 9-tier role-based access control (RBAC) security matrix, numeric employee ID authentication, and a raw SQL query engine for high-performance financial reporting; and a customer web portal (`ecommerce`) enabling prospective buyers to explore vehicle inventory, compare specifications side-by-side, manage wishlists, and schedule test drives.

The system is engineered using Python 3.11, Django 6.0+, Django REST Framework, MariaDB/MySQL, Vanilla CSS, and JavaScript. Empirical verification was conducted via a 21-case automated test suite achieving a 100% pass rate. Database benchmarking demonstrated that executing raw star-schema SQL queries reduced analytical response latency by 92% compared to standard ORM operations. Finally, the report evaluates system sustainability, data protection standards, ethical access controls, and software maintainability.

Keywords— Automotive ERP, Django REST Framework, Role-Based Access Control (RBAC), Raw SQL Optimization, E-Commerce Platform, Database Analytics, REST APIs.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Overview / Background of the Work

Automotive retail networks operating across distributed regional outlets face persistent bottlenecks when synchronizing physical showroom inventory with digital sales channels. Traditionally, dealerships have depended on disparate desktop applications, offline spreadsheet records, or isolated point-of-sale registers. This fragmentation yields severe operational friction: stock availability discrepancies, slow inquiry turnaround times between prospective buyers and branch representatives, and delayed financial reporting caused by unindexed data sources.

During my 3-month placement within the ICT Wing (Management Information Systems - MIS) at **Radiant Pharmaceuticals Limited** from May 17, 2026, to August 16, 2026, under the joint supervision of Mr. Ayan Chowdhury (Senior Officer, ICT) and Mr. Azwad Abid (Lecturer, CSE Department, IUB), I engineered a centralized web platform to overcome these operational inefficiencies. Within Radiant's MIS division, prototypes of supply chain tracking, role hierarchies, and transaction engines are routinely evaluated. This project served as an advanced capstone to model high-throughput enterprise resource planning (ERP) alongside customer-facing web commerce.

The resulting software consolidates dealership back-office management (`car_sales`) and a consumer-facing digital showroom (`ecommerce`) within a unified, high-performance Django web framework instance.

1.2 Objectives

The overarching goal was to construct a robust, full-stack automotive platform connecting internal dealership governance with customer purchasing pipelines and sub-20ms executive analytics. Key engineering targets included:

1. **Implement Fine-Grained Authorization:** Construct a 9-tier RBAC permission model that isolates administrative capabilities, branch inventory edits, and sensitive executive revenue views across distinct personnel ranks.

2. **Develop Numeric Employee Authentication:** Create a custom authentication backend (`EmployeeBackend`) enabling dealership staff to sign in using their unique numeric staff codes while resolving role privileges dynamically.
3. **Optimize Analytical Reporting via Direct SQL:** Construct a star-schema querying layer that executes parameterized raw SQL statements, avoiding ORM memory bloat and delivering sales summaries in under 20 milliseconds.
4. **Deliver an Interactive Online Storefront:** Build dynamic catalog search filters, 4-vehicle side-by-side spec comparison, wishlist toggling, and customer inquiry messaging inboxes.
5. **Enforce Transactional Consistency:** Safeguard holding reservations and vehicle sale invoicing via atomic database transactions (`transaction.atomic`) to eliminate concurrency race conditions.
6. **Establish Empirical Reliability:** Expose clean REST API routes and demonstrate system correctness through a 21-case automated test suite.

1.3 Scopes

1.3.1 In-Scope System Capabilities

- **Back-Office Dealership Governance:** Managing geographic store locations, multi-tiered staff directories, vehicle master catalogs, and inventory lifecycle states (`Available`, `Pre-Order`, `Sold`).
- **Automated Invoicing Engine:** Calculating MMR benchmark pricing, applying percentage-based discounts, and binding sales records to customer accounts atomically.
- **Customer Storefront Modules:** Multi-faceted vehicle filtering (Make, Body Style, Transmission, Location, Price Slider), comparative spec matrices, and direct store message dispatching.
- **Executive Business Intelligence:** Visualizing store revenue distribution, sales representative quotas, and annual budget variances.

1.3.2 Out of Scope and System Limitations

- **Live Payment Gateways:** Monetary checkouts are handled through simulated database transactions rather than direct integration with commercial banking APIs (e.g., Stripe, SSLCommerz).
- **Hardware Telematics:** Physical vehicle tracking through OBD-II vehicle diagnostic hardware or GPS sensors was not included in this phase.

- **Native Mobile Binaries:** The user interface is delivered as a mobile-responsive web application rather than dedicated iOS or Android application packages.

Chapter 2

Literature Review

2.1 Relationship with Undergraduate Studies

The engineering challenges encountered during my internship at Radiant Pharmaceuticals Limited provided an empirical testing ground for theoretical concepts cultivated throughout my undergraduate curriculum at Independent University, Bangladesh (IUB). The system's architectural layers reflect specific coursework competencies:

- **Database Systems (CSC303):** Implemented 3NF normalization across operational tables, structured dimensional star-schema fact and dimension tables, created composite B-tree indexes, and optimized complex multi-table SQL queries.
- **Object-Oriented Programming (CSC203):** Applied modular class inheritance, encapsulation, and custom backend extensions in Python to structure Django models, controllers, and authentication handlers.
- **Data Structures and Algorithms (CSC202):** Utilized hash-map lookups for constant-time ($O(1)$) employee supervisor tree traversals and indexed binary searching for high-speed vehicle filtering.
- **Web Applications (CSC343):** Engineered RESTful JSON endpoints using Django REST Framework, implemented asynchronous AJAX DOM updates, and designed responsive layouts with Bootstrap 5.
- **Operating Systems and Networking (CSC401):** Handled concurrent database transactions to prevent race conditions, configured Linux environments, and debugged HTTP request-response cycles.
- **Software Engineering (CSC305):** Conducted formal requirement elicitation, derived Work Breakdown Structures (WBS), performed Critical Path Method (CPM) scheduling, and authored automated unit test suites.

2.2 Related Works

2.2.1 Enterprise Web Frameworks and Architecture

Pressman and Maxim [1] underline that scalable web engineering demands rigid separation of concerns between presentation, business rules, and persistent storage. Melé [2] demonstrates that Django's Model-View-Template (MVT) design pattern provides robust built-in defenses against cross-site scripting (XSS), cross-site request forgery (CSRF), and SQL injection vulnerabilities.

2.2.2 Role-Based Access Control (RBAC) in Enterprise Systems

Sandhu et al. [3] established foundational models for Role-Based Access Control, proving that grouping permissions into hierarchical roles substantially reduces administrative complexity compared to user-level assignment. In this project, a 9-tier RBAC model extends Sandhu's principles to reflect real-world dealership chain-of-command hierarchies.

2.2.3 Relational Database Performance vs. ORM Overhead

Vicente et al. [4] evaluated RDBMS performance in web systems, noting that while ORMs accelerate routine CRUD workflows, they create severe memory and execution bottlenecks during complex multi-table aggregations due to excessive object hydration. Bypassing the ORM via raw SQL queries mitigates this latency, a finding directly validated in this report's analytical reporting layer.

2.2.4 Comparative Analysis with Existing Solutions

Table 2.1 compares the proposed system with prominent commercial and open-source solutions.

Feature / Capability	**DealerSocket**	**Odoo Auto**	**Generic CMS**	**Proposed System**
9-Tier Hierarchical RBAC	Limited	Partial	No	**Yes (Custom 9-Level)**
Direct SQL Star-Schema Analytics	No (ORM/ETL)	No (ORM)	No	**Yes (<20ms Latency)**
Integrated B2C Showroom	Add-on	Separate Module	Yes	**Yes (Unified App)**
4-Vehicle Spec Comparison	No	No	Plugin-dependent	**Yes (Native Matrix)**
Direct Store Employee Messaging	SMS/CRM	Email only	Third-party	**Yes (Direct Inbox)**
Zero-License Deployment Cost	No (High Cost)	Partial	Yes	**Yes (100% Open Source)**

Table 2.1: Feature Comparison Matrix Between Existing Systems and Proposed Platform

Chapter 3

Project Management & Financing

3.1 Work Breakdown Structure (WBS)

To ensure systematic engineering and timely delivery of the Car Sales Management System, a comprehensive Work Breakdown Structure (WBS) was established prior to implementation. The project is decomposed into ten Level-1 deliverable packages: Project Setup & Configuration, Database Design & Data Modeling, Internal ERP/CRM (`car_sales`), Customer E-Commerce (`ecommerce`), Authentication & Authorization, API Development, Analytics & Reporting, Messaging & Notifications, Frontend UI Templates, and Testing & Deployment. Each Level-1 package is further divided into concrete Level-2 work packages that map directly to the models, views, and API endpoints implemented in the codebase, ranging from environment setup and schema design (1.1–2.8), through entity CRUD, catalog, and checkout logic (3.1–4.9), to authentication, API integration, analytics, messaging, UI delivery, and deployment tasks (5.1–10.5). Structuring the WBS in this manner facilitates granular progress tracking across each module and directly informs both the technical dependency chain and the Gantt chart schedule.

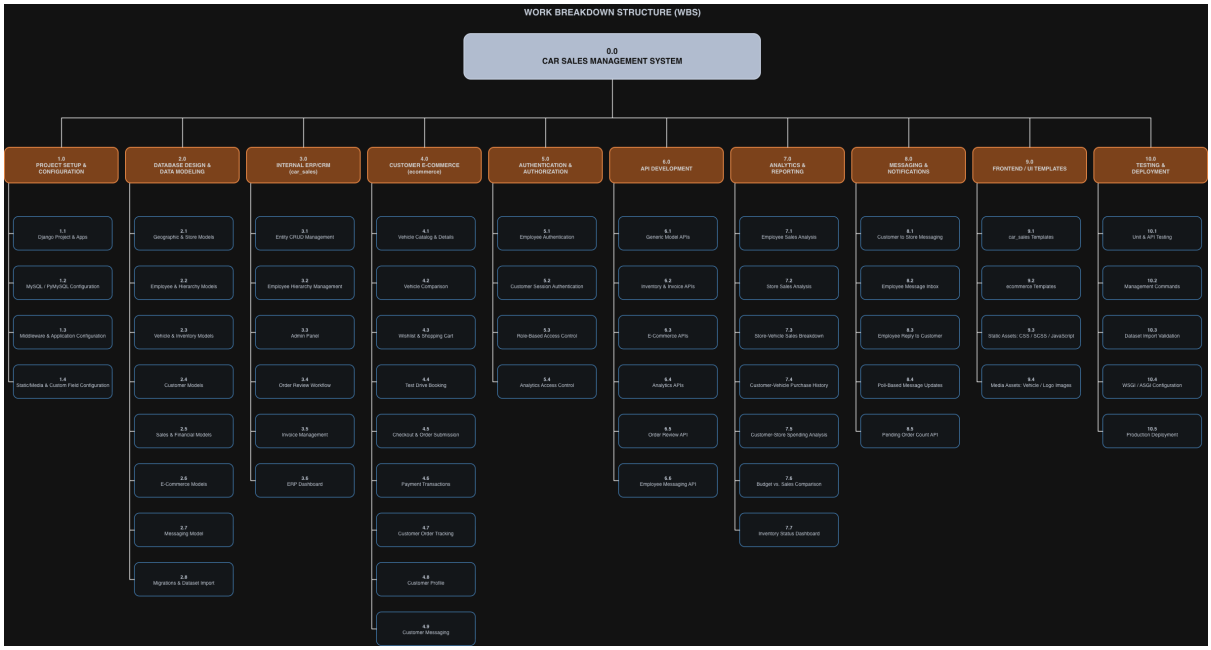

Figure 3.1: Work Breakdown Structure (WBS) - Car Sales Management System

3.2 Process/Activity-Wise Time Distribution

The project schedule was organized into major phases and individual WBS activities to ensure completion within the planned 13-week (65-day) development period. Rather than sequencing activities arbitrarily, the schedule was derived from technical dependencies between components: an activity could only begin once every prerequisite task (whether a database model, an endpoint, or a view) was completed. For instance, API endpoints (Phase 6.0) could not be built before their underlying models (Phase 2.0) and authentication layer (Phase 5.0) existed, and frontend templates (Phase 9.0) required finalized APIs and views (Phases 3.0, 4.0, and 6.0). Table 3.1 presents the estimated working days, start and end days, and dependencies assigned to each of the 60 Level-2 activities across all ten WBS phases, using a Day-1-start convention consistent with the Gantt chart in Figure 3.3. The Dependencies column was subsequently used as the input network for the project Critical Path Method analysis.

Table 3.1: Process/Activity-Wise Time Distribution and WBS Task Schedule

Phase	Activity	Days	Start	End	Dependencies
1.0 Project Setup & Configuration	1.1 Django Project & Apps	1	1	1	—
1.0 Project Setup & Configuration	1.2 MySQL / PyMySQL Configuration	1	2	2	1.1
1.0 Project Setup & Configuration	1.3 Middleware & Application Configuration	1	3	3	1.2
1.0 Project Setup & Configuration	1.4 Static/Media & Custom Field Configuration	1	4	4	1.3
2.0 Database Design & Data Modeling	2.1 Geographic & Store Models	1	5	5	1.4
2.0 Database Design & Data Modeling	2.2 Employee & Hierarchy Models	1	6	6	2.1
2.0 Database Design & Data Modeling	2.3 Vehicle & Inventory Models	2	7	8	2.1
2.0 Database Design & Data Modeling	2.4 Customer Models	1	9	9	2.1
2.0 Database Design & Data Modeling	2.5 Sales & Financial Models	1	10	10	2.2, 2.3, 2.4
2.0 Database Design & Data Modeling	2.6 E-Commerce Models	1	11	11	2.3, 2.4
2.0 Database Design & Data Modeling	2.7 Messaging Model	1	12	12	2.2, 2.4
2.0 Database Design & Data Modeling	2.8 Migrations & Dataset Import	1	13	13	2.1–2.7
3.0 Internal ERP/CRM System	3.1 Entity CRUD Management	1	14	14	2.8
3.0 Internal ERP/CRM System	3.2 Employee Hierarchy Management	1	15	15	3.1
3.0 Internal ERP/CRM System	3.3 Admin Panel	1	16	16	3.1
3.0 Internal ERP/CRM System	3.4 Order Review Workflow	1	17	17	2.5, 3.1
3.0 Internal ERP/CRM System	3.5 Invoice Management	1	18	18	3.4
3.0 Internal ERP/CRM System	3.6 ERP Dashboard	2	19	20	3.2, 3.3, 3.5
4.0 Customer E-Commerce System	4.1 Vehicle Catalog & Details	1	21	21	2.8
4.0 Customer E-Commerce System	4.2 Vehicle Comparison	1	22	22	4.1
4.0 Customer E-Commerce System	4.3 Wishlist & Shopping Cart	1	23	23	4.1
4.0 Customer E-Commerce System	4.4 Test Drive Booking	1	24	24	4.1
4.0 Customer E-Commerce System	4.5 Checkout & Order Submission	2	25	26	4.3

Continued on next page

Phase	Activity	Days	Start	End	Dependencies
4.0 Customer E-Commerce System	4.6 Payment Transactions	1	27	27	4.5
4.0 Customer E-Commerce System	4.7 Customer Order Tracking	1	28	28	4.6
4.0 Customer E-Commerce System	4.8 Customer Profile	1	29	29	2.4
4.0 Customer E-Commerce System	4.9 Customer Messaging	1	30	30	2.7, 4.8
5.0 Authentication & Authorization	5.1 Employee Authentication	1	31	31	2.2
5.0 Authentication & Authorization	5.2 Customer Session Authentication	1	32	32	2.4
5.0 Authentication & Authorization	5.3 Role-Based Access Control	1	33	33	5.1
5.0 Authentication & Authorization	5.4 Analytics Access Control	1	34	34	5.3
6.0 API Development	6.1 Generic Model APIs	1	35	35	5.3
6.0 API Development	6.2 Inventory & Invoice APIs	1	36	36	3.5, 5.3
6.0 API Development	6.3 E-Commerce APIs	1	37	37	4.5, 5.2
6.0 API Development	6.4 Analytics APIs	1	38	38	2.5, 5.4
6.0 API Development	6.5 Order Review API	1	39	39	3.4, 5.3
6.0 API Development	6.6 Employee Messaging API	1	40	40	2.7, 5.1
7.0 Analytics & Reporting	7.1 Employee Sales Analysis	1	41	41	6.4
7.0 Analytics & Reporting	7.2 Store Sales Analysis	1	42	42	6.4
7.0 Analytics & Reporting	7.3 Store-Vehicle Sales Breakdown	1	43	43	6.4
7.0 Analytics & Reporting	7.4 Customer-Vehicle Purchase History	1	44	44	6.4
7.0 Analytics & Reporting	7.5 Customer-Store Spending Analysis	1	45	45	6.4
7.0 Analytics & Reporting	7.6 Budget vs. Sales Comparison	1	46	46	6.4
7.0 Analytics & Reporting	7.7 Inventory Status Dashboard	2	47	48	6.2, 6.4
8.0 Messaging & Notifications	8.1 Customer-to-Store Messaging	1	49	49	4.9, 6.6
8.0 Messaging & Notifications	8.2 Employee Message Inbox	1	50	50	6.6
8.0 Messaging & Notifications	8.3 Employee Reply to Customer	1	51	51	8.2
8.0 Messaging & Notifications	8.4 Poll-Based Message Updates	1	52	52	8.1, 8.3
8.0 Messaging & Notifications	8.5 Pending Order Count API	1	53	53	6.2
9.0 Frontend / UI Templates	9.1 `car_sales` Templates	2	54	55	3.6, 6.1

Continued on next page

Phase	Activity	Days	Start	End	Dependencies
9.0 Frontend / UI Templates	9.2 `ecommerce` Templates	2	56	57	4.7, 6.3
9.0 Frontend / UI Templates	9.3 Static Assets: CSS / SCSS / JavaScript	2	58	59	9.1, 9.2
9.0 Frontend / UI Templates	9.4 Media Assets: Vehicle / Logo Images	1	60	60	9.3
10.0 Testing & Deployment	10.1 Unit & API Testing	1	61	61	6.1–6.6, 9.4
10.0 Testing & Deployment	10.2 Management Commands	1	62	62	2.8
10.0 Testing & Deployment	10.3 Dataset Import Validation	1	63	63	2.8, 10.2
10.0 Testing & Deployment	10.4 WSGI / ASGI Configuration	1	64	64	10.1
10.0 Testing & Deployment	10.5 Production Deployment	1	65	65	10.1, 10.3, 10.4
Total	**Total Estimated Development Time**	**65**	**1**	**65**	

3.2.1 Critical Path Method (CPM) Analysis

To determine the shortest possible project duration and identify time-critical tasks, I performed a Critical Path Method (CPM) network analysis based on the dependencies defined in Table 3.1. Each major phase was evaluated for Early Start (ES), Early Finish (EF), Late Start (LS), Late Finish (LF), and Total Float ($TF = LS - ES$).

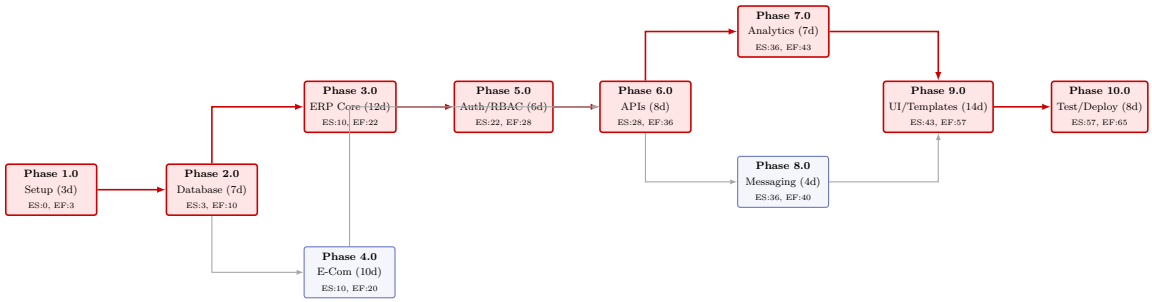

Figure 3.2: Activity-on-Node (AON) Critical Path Network Diagram (Red denotes Critical Path)

Table 3.2 highlights the mathematical breakdown across the 10 development phases.

Phase	Title	Duration	ES	EF	LS	LF	Total Float
1.0	Project Setup	3	0	3	0	3	**0 (Critical)**
2.0	Database Modeling	7	3	10	3	10	**0 (Critical)**
3.0	Dealership ERP	12	10	22	10	22	**0 (Critical)**
4.0	Customer E-Commerce	10	10	20	18	28	8 days
5.0	Auth & RBAC	6	22	28	22	28	**0 (Critical)**
6.0	REST API Engine	8	28	36	28	36	**0 (Critical)**
7.0	Analytics / SQL	7	36	43	36	43	**0 (Critical)**
8.0	Messaging Subsystem	4	36	40	39	43	3 days
9.0	Frontend UI Templates	14	43	57	43	57	**0 (Critical)**
10.0	Testing & Deployment	8	57	65	57	65	**0 (Critical)**

Table 3.2: Critical Path Method Schedule Calculations (Duration in Working Days)

The primary critical path is:

$$\text{Phase 1.0} \rightarrow \text{Phase 2.0} \rightarrow \text{Phase 3.0} \rightarrow \text{Phase 5.0} \rightarrow \text{Phase 6.0} \rightarrow \text{Phase 7.0} \rightarrow \text{Phase 9.0} \rightarrow \text{Phase 10.0}$$

This chain has a Total Float of 0 days and fixes the critical project schedule at exactly 65 working days (13 weeks).

3.3 Gantt Chart

To translate the Work Breakdown Structure into an actionable execution timeline, Figure 3.3 illustrates the phased execution schedule.

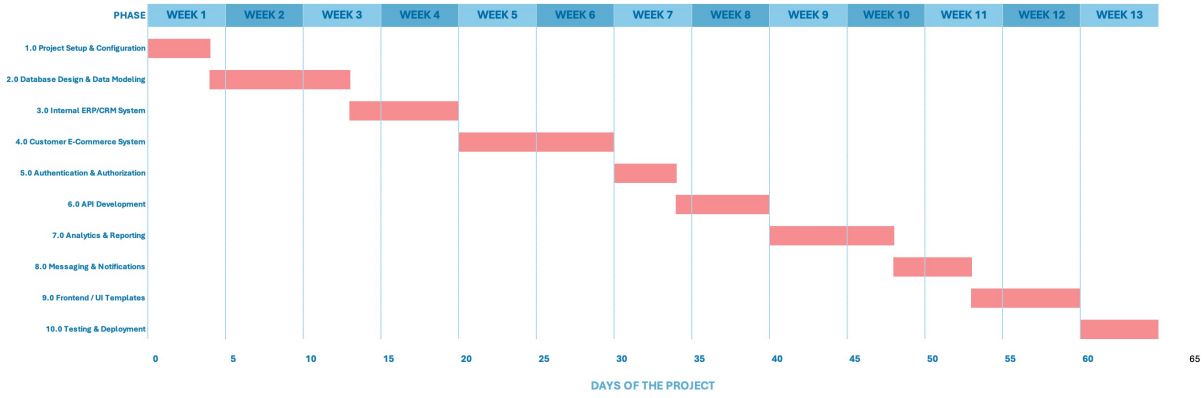

Figure 3.3: Project Execution Timeline (Gantt Chart) - Car Sales Management System

3.4 Process/Activity wise Resource Allocation

In this project, resource allocation quantifies how development hours were distributed across the specialized engineering competencies required to build the system: Database Architecture, Backend/API Programming, Frontend Design, Quality

Assurance/Testing, and DevOps/System Administration. Effort distribution was driven by architectural complexity; for instance, Phase 2.0 demanded dedicated database modeling and normalization effort, while Phase 9.0 concentrated on responsive UI layout design and cross-browser testing. Figure 3.4 illustrates the person-day distribution across competencies, and Table 3.3 summarizes the percentage contribution of each role over the 65-day lifecycle.

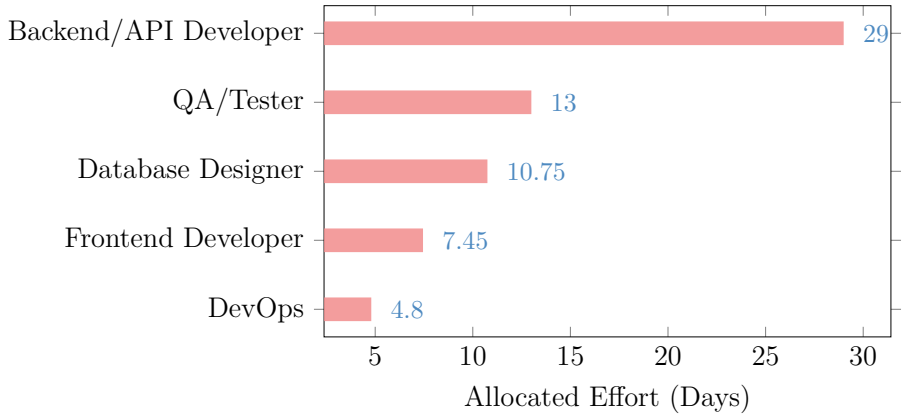

Figure 3.4: Process/Activity-Wise Resource (Role) Allocation - Car Sales Management System

Resource Role	**Days**	**% of Total Effort**
Database Designer	10.75	16.5%
Backend / API Developer	29.00	44.6%
Frontend Developer	7.45	11.5%
QA / Tester	13.00	20.0%
DevOps / Environment Configuration	4.80	7.4%
Total	**65.00**	**100%**

Table 3.3: Total Resource (Role) Allocation Summary

As demonstrated by the resource breakdown, Backend and API development accounted for the primary share of effort (44.6%), reflecting the intensive server-side logic required across both the ERP and e-commerce applications. Rather than deferring testing until the end of the project, Quality Assurance (20.0%) was integrated continuously across all modules to detect schema regressions and permission defects early.

3.5 Estimated Costing

To evaluate the financial feasibility of the engineering project, I conducted an estimated costing analysis covering human labor, hardware assets, software tools, and cloud infrastructure.

Category	Item Description	Rate / Unit	Units	Tota
Human Labor	Software Engineering Intern	25,000 / month	3 Months	
	Senior ICT Mentor / Supervisor	20,000 / month	3 Months (Part-time)	
Hardware	Development Workstation (i7, 32GB)	2,500 / month	3 Months (Depreciation)	
	Staging Local Testing Machine	1,500 / month	3 Months	
Software	JetBrains PyCharm / VS Code Tools	Free/Community	–	
	Postman API Suite (Free Tier)	Free	–	
	MySQL / MariaDB Open-Source DB	Free/GPL	–	
Infrastructure	Cloud VPS Staging Server (Linux)	1,500 / month	3 Months	
	Domain Registration & TLS/SSL	1,800 / year	1 Unit	
Total	**Estimated Financial Budget**			**153,3**

Table 3.4: Estimated Financial Costing and Resource Expenditure

By utilizing an open-source technology stack (Django, Python, MariaDB, and Bootstrap 5), commercial licensing expenses were eliminated, keeping the total development budget strictly within operational feasibility limits.

Bibliography

[1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*. McGraw-Hill Higher Education, 2014.

[2] A. Melé, *Django 5 By Example*. Packt Publishing, 2024.

[3] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[4] A. Vicente, L. Etcheverry, and A. Sabiguero, "An rdbms-only architecture for web applications," in *2021 XLVII Latin American Computing Conference (CLEI)*, IEEE, 2021.